Detailed API Testing Guide

Step-by-Step Instructions for Testing Your FastAPI Backend

📋 Table of Contents

Prerequisites

✅ Checklist Before Testing

- [ ] Server is running (you see "Uvicorn running on http://0.0.0.0:8000")
- [ ] You have an Excel file ready for testing
- [ ] Browser is open (Chrome, Edge, or Firefox)
- [ ] You're in the project directory

Starting the Server

Step 1: Open Terminal/PowerShell

Windows: - Press `Win + X` - Select "Windows PowerShell" or "Terminal"

Step 2: Navigate to Project Directory

```
cd E:\JET\SivaSakthiInvoiceGenerator
```

Step 3: Start the Server

```
uvicorn api_server:app --reload --host 0.0.0.0 --port 8000
```

Step 4: Verify Server Started

You should see:

```
 INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [3484] using WatchFiles
INFO:     Started server process [3484]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

✅ Server is ready!

Method 1: Testing with Swagger UI (Easiest)

This is the **easiest and most visual** way to test your API.

Step 1: Open Swagger UI

In your browser, go to:

```
http://localhost:8000/docs
```

What you'll see: - A page titled "FastAPI" - List of all available endpoints - Green/Blue/Orange buttons for different HTTP methods

---

Step 2: Test Health Check Endpoint

2.1: Find the Health Check Endpoint

Scroll to find:

```
GET /health
Health Check
```

2.2: Click on it to expand

Click the green `GET /health` button.

2.3: Click "Try it out"

You'll see a blue button that says **"Try it out"** - click it.

2.4: Click "Execute"

A blue button **"Execute"** will appear - click it.

2.5: See the Response

Scroll down to see:

Response Code:

```
200
```

Response Body:

```
{
  "status": "healthy",
  "timestamp": "2025-11-26T10:30:00.000000",
  "agents": {
    "tax_agent": "available",
    "bill_agent": "available"
  }
}
```

✅ **Success!** Your API is working!

Step 3: Test Get Default Bill Invoice

3.1: Find the Endpoint

Scroll to:

```
 GET /api/bill/default
Get Default Bill Invoice State
```

3.2: Click to expand → Click "Try it out" → Click "Execute"

3.3: Check Response

Response Code: `200`

Response Body:


```
{
  "success": true,
  "state": {
    "freight_bill": {
      "series_no": "",
      "bill_date": "",
      "runs": [],
      "total": 0.0
    }
  },
  "message": "Default bill invoice state"
}
```


✅ **Success!** Default state is returned correctly!

Step 4: Test File Upload and Processing

This is the most important test!

4.1: Find the Endpoint

Scroll to:

```
POST /api/bill/process
Process Bill Invoice
```

4.2: Click to expand → Click "Try it out"

4.3: Fill in the Form

You'll see three fields:

Field 1: file - Click **"Choose File"** - Select your Excel file (e.g., `4100900450I00760800011EXY.xlsx`)

Field 2: state - Leave the default JSON or paste this:

```
{
  "freight_bill": {
    "series_no": "",
    "bill_date": "",
    "runs": [],
    "total": 0.0
  }
}
```

Field 3: command - Type: `add all details from dataset to invoice`

4.4: Click "Execute"

Wait 2-5 seconds for processing...

4.5: Check Response

Response Code: `200`

Response Body:

```
{
  "success": true,
  "state": {
    "freight_bill": {
      "series_no": "",
      "bill_date": "",
      "to_party": {
        "name": "COROMANDEL INTERNATIONAL LIMITED"
      },
      "runs": [
        {
          "date": "21-10-2025",
          "truck_no": "AP39TU3699",
          "lr_no": "",
          "dc_qty_mt": 34.72,
          "gr_qty_mt": 34.44,
          "rate": 710,
          "line_total": 24452.4
        },
        // ... more runs
      ],
      "total": 702261.0
    }
  },
  "message": "✓ Agentic operation complete: ADD"
}
```


✅ **Success!** Your file was processed and data extracted!

Check: - [ ] `success: true` - [ ] `runs` array has data - [ ] Client name is extracted - [ ] Totals are calculated

---

Step 5: Test Export to HTML

5.1: Find the Endpoint

```
POST /api/bill/export
Export Bill Invoice
```

5.2: Click to expand → Click "Try it out"

5.3: Fill in Request Body

Copy the **entire state** from the previous response and paste it into the request body:

```
{
  "state": {
    "freight_bill": {
      "series_no": "TEST-001",
      "bill_date": "26-11-2025",
      "to_party": {
        "name": "COROMANDEL INTERNATIONAL LIMITED"
      },
      "runs": [
        {
          "date": "21-10-2025",
          "truck_no": "AP39TU3699",
          "lr_no": "",
          "dc_qty_mt": 34.72,
          "gr_qty_mt": 34.44,
          "rate": 710,
          "line_total": 24452.4
        }
      ],
      "total": 24452.4
    }
  },
  "format": "html"
}
```


5.4: Click "Execute"

5.5: Check Response

Response Code: `200`

Response Body:


```
{
  "html": "<html><head>...</head><body>...</body></html>"
}
```


✅ **Success!** HTML invoice generated!

Method 2: Testing with PowerShell/CMD

Test 1: Health Check

PowerShell:

```
Invoke-RestMethod -Uri "http://localhost:8000/health" -Method Get | ConvertTo-Json
```

Expected Output:

```
{
  "status": "healthy",
  "agents": {
    "tax_agent": "available",
    "bill_agent": "available"
  }
}
```

Test 2: Get Default State

PowerShell:

```
Invoke-RestMethod -Uri "http://localhost:8000/api/bill/default" -Method Get | ConvertTo-Json -Depth 10
```

Expected Output:

```
{
  "success": true,
  "state": {
    "freight_bill": {
      "runs": []
    }
  }
}
```

Test 3: Process File Upload

PowerShell:

```
# Set file path
$filePath = "E:\JET\SivaSakthiInvoiceGenerator\your_file.xlsx"

# Create form data
$form = @{
    file = Get-Item -Path $filePath
    state = '{"freight_bill":{"runs":[]}}'
    command = "add all details from dataset to invoice"
}

# Send request
$response = Invoke-RestMethod -Uri "http://localhost:8000/api/bill/process" -Method Post -Form $form

# Display result
$response | ConvertTo-Json -Depth 10
```

Expected Output:

```
{
  "success": true,
  "state": {
    "freight_bill": {
      "runs": [
        // ... extracted data
      ]
    }
  },
  "message": "✓ Agentic operation complete: ADD"
}
```

Method 3: Testing with Postman

Step 1: Install Postman

Download from: https://www.postman.com/downloads/

Step 2: Create New Request

1. Click **"New"** → **"HTTP Request"**
2. Set method to **GET**
3. Enter URL: `http://localhost:8000/health`
4. Click **"Send"**

Step 3: Test File Upload

1. Create new request
2. Set method to **POST**
3. Enter URL: `http://localhost:8000/api/bill/process`
4. Go to **"Body"** tab
5. Select **"form-data"**
6. Add fields:
7. Key: `file`, Type: **File**, Value: Select your Excel file
8. Key: `state`, Type: **Text**, Value: `{"freight_bill":{"runs":[]}}`

9. Key: `command`, Type: **Text**, Value: `add all details from dataset to invoice`
10. Click **"Send"**

Step 4: Check Response

Look at the response panel below: - Status: `200 OK` - Body: JSON with extracted data

---

Method 4: Testing with Python Script

Create Test Script

Create `test_api.py`:

```
import requests
import json

# Test 1: Health Check
print("Test 1: Health Check")
response = requests.get("http://localhost:8000/health")
print(f"Status: {response.status_code}")
print(f"Response: {response.json()}\n")

# Test 2: Get Default State
print("Test 2: Get Default Bill Invoice State")
response = requests.get("http://localhost:8000/api/bill/default")
result = response.json()
print(f"Status: {response.status_code}")
print(f"Success: {result['success']}")
print(f"Runs count: {len(result['state']['freight_bill']['runs'])}\n")

# Test 3: Process File Upload
print("Test 3: Process File Upload")
file_path = r"E:\JET\SivaSakthiInvoiceGenerator\your_file.xlsx"

with open(file_path, 'rb') as f:
    files = {'file': ('data.xlsx', f, 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet')}
    data = {
        'state': json.dumps({"freight_bill": {"runs": []}}),
        'command': 'add all details from dataset to invoice'
    }

    response = requests.post("http://localhost:8000/api/bill/process",
files=files, data=data)
    result = response.json()

    print(f"Status: {response.status_code}")
    print(f"Success: {result['success']}")
    print(f"Message: {result['message']}")
    print(f"Runs extracted: {len(result['state']['freight_bill']['runs'])}")
    print(f"Total: {result['state']['freight_bill']['total']}")
```

Run Test Script

```
python test_api.py
```

Expected Output:

```
 Test 1: Health Check
Status: 200
Response: {'status': 'healthy', 'agents': {...}}

Test 2: Get Default Bill Invoice State
Status: 200
Success: True
Runs count: 0

Test 3: Process File Upload
Status: 200
Success: True
Message: ✓ Agentic operation complete: ADD
Runs extracted: 28
Total: 702261.0
```

Complete Test Scenarios

Scenario 1: Bill Invoice - Full Workflow

Step 1: Get Default State

```
GET http://localhost:8000/api/bill/default
```

Step 2: Upload File and Extract

```
POST http://localhost:8000/api/bill/process
- file: your_data.xlsx
- command: "add all details from dataset to invoice"
```

Step 3: Fill Empty LR Numbers

```
POST http://localhost:8000/api/bill/process
- state: (previous response state)
- command: "fill all empty LR No with values 40, 42, 44"
```

Step 4: Export to HTML

```
POST http://localhost:8000/api/bill/export
- state: (previous response state)
- format: "html"
```

Scenario 2: Tax Invoice - Full Workflow

Step 1: Get Default State

```
GET http://localhost:8000/api/tax/default
```

Step 2: Set Invoice Details

```
POST http://localhost:8000/api/tax/process
- state: (default state)
- command: "set invoice number to INV-2025-001 and invoice date to 26-11-2025"
```

Step 3: Add Items from File

```
POST http://localhost:8000/api/tax/process
- file: items_data.xlsx
- state: (previous response state)
- command: "add all details from dataset"
```

Step 4: Export to HTML

```
POST http://localhost:8000/api/tax/export
- state: (previous response state)
- format: "html"
```

What Success Looks Like

✅ Health Check Success

```
{
  "status": "healthy",
  "agents": {
    "tax_agent": "available",
    "bill_agent": "available"
  }
}
```

✅ File Processing Success

```
{
  "success": true,
  "state": {
    "freight_bill": {
      "runs": [/* 28 items */],
      "total": 702261.0
    }
  },
  "message": "✓ Agentic operation complete: ADD"
}
```

✅ Export Success (HTML)

```
{
  "html": "<html>...</html>"
}
```

✅ Export Success (PDF)

- Response: Binary PDF file
- Content-Type: `application/pdf`
- File downloads automatically

Troubleshooting

Issue: "Connection refused"

Symptom:

```
Failed to connect to localhost:8000
```

Solution: - Check if server is running - Look for "Uvicorn running" message in terminal - Restart server if needed

---

Issue: "404 Not Found"

Symptom:

```
{"detail":"Not Found"}
```

Solution: - Check URL spelling - Use `/api/bill/process` not `/api/bills/process` - Check Swagger UI for correct endpoint names

---

Issue: "422 Unprocessable Entity"

Symptom:

```
{"detail":[{"loc":["body","state"],"msg":"field required"}]}
```

Solution: - Check all required fields are provided - Verify JSON format is correct - Use Swagger UI to see required fields

---

Issue: "500 Internal Server Error"

Symptom:

```
{"detail":"Processing failed: ..."}
```

Solution: - Check server terminal for error details - Verify Excel file format is correct - Check OpenAI API key is set - Look at error message in response

---

Quick Reference

All Endpoints

Method	Endpoint	Purpose
GET	`/health`	Check server status
GET	`/api/tax/default`	Get default tax invoice
POST	`/api/tax/process`	Process tax invoice
POST	`/api/tax/export`	Export tax invoice
GET	`/api/bill/default`	Get default bill invoice
POST	`/api/bill/process`	Process bill invoice
POST	`/api/bill/export`	Export bill invoice

Base URLs

- **Swagger UI:** http://localhost:8000/docs
- **ReDoc:** http://localhost:8000/redoc
- **API Base:** http://localhost:8000

Summary Checklist

After testing, you should have verified:

- [ ] Health check returns "healthy"
- [ ] Can get default state for both invoice types

- [ ] Can upload Excel file and extract data
- [ ] Data is extracted correctly (check runs count)
- [ ] Totals are calculated correctly
- [ ] Can process commands without file
- [ ] Can export to HTML
- [ ] Can export to PDF (optional)

You're all set! 🚀

If all tests pass, your API is ready for production and your frontend developer can start integrating!